

Siddhant Kumar Upmanyu

Senior Systems Architect

sku20.dev github.com/sku0x20 linkedin.com/in/sku20 siddhantkumarupmanyu@gmail.com +91 8383056329

PROFILE SUMMARY

4+ years of full ownership over backend and cloud infrastructure at an IoT smart home company — from protocol design through production operations. Design at the protocol level when the problem calls for it: custom binary transports over UDP, kernel-level traffic routing, nftables packet reflection. Publish the tools that come out of that work: a Kotlin microservice framework on Maven Central, a Go assertion library, a Helm CLI in Rust, a custom Linux distro. Treat TDD as non-negotiable across every language and project.

SKILLS

LANGUAGES

Kotlin, Java, Go, Rust, Zig, C, Shell

FRAMEWORKS & PROTOCOLS

Spring Boot, Helidon SE, gRPC, Protocol Buffers

INFRASTRUCTURE

Kubernetes (EKS), Terraform, Docker, GitHub Actions, Linux (Debian/Ubuntu)

DATABASES

ClickHouse, MongoDB, PostgreSQL, Redis

OBSERVABILITY

Loki, Prometheus, Grafana, async-profiler, JMC, VisualVM, Flame Graphs

TESTING

TDD (London style), Pact (Consumer-Driven Contracts), Playwright, Testcontainers

NETWORKING

TCP/UDP socket programming, iptables, nftables, NAT traversal

WORK EXPERIENCE

eGlu Smart Homes (WiZNSystems)

Bengaluru, India · 4+ years

Senior Systems Architect

Apr 2026 – Present

Backend & Infrastructure Engineer

Mar 2022 – Mar 2026

Architecture & API Design

- **API Contract Ownership:** Own the API contract suite for the entire platform — 30+ specs covering device provisioning, scene/rule automation, OTA, smart lock integrations, and real-time device status; app and firmware teams build against these before a line of code is written.
- **Technical Direction:** Lead technical specification for the unified backend and cloud infrastructure stack, security standards, and data contracts.

IoT Protocol & Real-time Systems

- **Custom Protocol Design:** Designed and implemented the custom binary protocol (UDP/TCP) between IoT hubs and cloud — packet handling, NAT hole punching, hub online/offline state machine, connection tracking, race condition handling.
- **Kernel-Level Networking:** Pushed network work to the kernel when needed — nftables UDP echo server for NAT traversal (packet reflection without userspace), IP rate limiting via per-source dynamic sets at prerouting priority.
- **Kernel Tuning:** Tuned kernel parameters for high-throughput IoT workloads — socket buffers, IP forwarding, connection tracking table size.

Platform & Automation Engine

- **Automation Engine:** Built the scene and rule automation engine — multi-hub scenes, nested fragment handling for large payloads, scene sync, IFTTT-style cloud rules, scheduling.
- **Provisioning System:** Built the device provisioning system from scratch (extracted from monolith) — hub setup, node validation, WiFi flow rewrite, replacement flow for failed nodes.
- **Third-Party Integrations:** Integrated Google Home, Alexa, and Yale smart locks (cloud-to-cloud) into the platform.
- **Controlled Rollouts:** Built a feature flags system for controlled rollouts.
- **Contract Testing:** Introduced Consumer-Driven Contract Testing (Pact) for firmware delivery APIs — full CI pipeline: publish pact → can-i-deploy → release → mark deployed.

Performance Engineering

- **Storage Optimization:** ClickHouse migration — rewrote core ingestion pipeline in Go, achieved **95%+ storage reduction** (90 GB → 3.7 GB, 25 GB → 780 MB) via schema design and specialized codecs.
- **JVM Performance:** Profiled with async-profiler and flame graphs — eliminated CPU spikes from per-packet object allocations using ThreadLocal; tuned GC for throughput.
- **Zero-Downtime Deploys:** Blue-green deploys via iptables — flushed conntrack for immediate cut-over, applied rules to both PREROUTING and OUTPUT chains for internal routing.
- **Error Handling:** Implemented structured ThreadPool error handling after debugging silently swallowed exceptions in async paths (no logs, no traces — the bug that inspired [the blog post](#)).

Infrastructure & Reliability

- **Observability Overhaul:** Migrated disk-based logging to Loki; built Prometheus + Grafana monitoring — significantly reduced MTTR.
- **Cloud Migration:** Architected EKS migration using Terraform; evaluated HashiCorp Nomad first, selected EKS.
- **Security:** Implemented TLS client authentication with a custom CA and certificate revocation list (CRL) in Go.
- **Systems Administration:** Maintained production Linux systems — kernel tuning, live Debian/Ubuntu upgrades, tcpdump, strace, sar, iotop.
- **CI/CD:** Built and maintained GitHub Actions pipelines for automated testing, building, and deployment.

Codebase & Quality

- **Framework Migration:** Migrated core monolith (Spring 4 / Java) to Spring Boot 2.7+ / Kotlin — 100% TDD throughout to prevent regressions.
- **Microservices:** Decoupled monolith into gRPC microservices.
- **Technical Hiring:** Led technical recruitment — designed coding challenges, conducted interviews.

EDUCATION

BCA from Maharaja Agrasen Himalayan Garhwal University · 2019–2022

OPEN SOURCE

stopgap [Kotlin](#) · [Maven Central](#)

Microservice framework on Helidon SE (Nima) + Project Loom. Custom compile-time DI via KSP — no runtime reflection. Three-tier testing: unit → in-process integration server → Docker E2E via Testcontainers. Gradle plugin that wires the full build pipeline. Published at `dev.sku20.stopgap:*:2.8.0`.

assertgo [Go](#)

Type-safe assertion library with generics. Fluent API, chainable `Not()`, custom matchers, zero dependencies. v1.0.0.

hrh [Rust](#)

Helm Release Helper — reads a YAML release declaration, invokes `helm upgrade --install`. Diff mode, `--atomic` rollback, chart versioning. Installable via `cargo install`.

avoid [Shell](#)

Minimal Linux distribution based on Void Linux, built for servers and recovery disks. Ships `.img.gz` and `.qcow2` images via GitHub Actions releases.

c_oop [C](#)

OOP and London-style TDD in pure C. Polymorphism via interface structs, heap-allocated objects with constructors. v5.0.0, CI.

PUBLICATIONS

Connection-Agnostic Presence Tracking for Stateless Distributed Backends [Preprint](#) · [Zenodo](#)

Redis-based architecture for tracking IoT device online/offline status across stateless distributed backends — sorted sets scored by expiry deadlines, throttled batch writes, asymmetric fault-tolerance guarantees ensuring offline transitions are never lost. Derives worst-case detection latency bounds. v1.3.0.